

Chapter 5

OOPS Basics: Functions, Constructors & the this Keyword

1. What Is a Function (Method) in Java?

A function (called a method in Java) is a block of code that performs a specific task. It helps make code reusable, organized, and easier to read.

```
returnType methodName(parameters) {  
    // method body  
    return value;  
}
```

Components

- `returnType` — the data type of the value the method returns (or void if it returns nothing).
- `methodName` — the name of the method (follows camelCase naming).
- `parameters` — optional input values passed to the method.
- `method body` — the code that runs when the method is called.

2. Methods Without Parameters

These methods don't take any inputs when called.

Example — Print a Message

```
public class Example1 {  
    static void sayHello() {  
        System.out.println("Hello, World!");  
    }  
    public static void main(String[] args) {  
        sayHello();  
    }  
}
```

Example — Display Current Year

```
public class Example2 {  
    static void showYear() {  
        int year = 2025;  
        System.out.println("The current year is " + year);  
    }  
    public static void main(String[] args) {  
        showYear();  
    }  
}
```

3. Methods With Parameters

These methods take input values (parameters) when called.

```
returnType methodName(type param1, type param2, ...) {  
    // method body  
}
```

Example — Addition Using Parameters

```
public class Example4 {  
    static void addNumbers(int a, int b) {  
        int sum = a + b;  
        System.out.println("Sum = " + sum);  
    }  
    public static void main(String[] args) {  
        addNumbers(5, 7);  
    }  
}
```

Example — Area of a Rectangle

```
public class Example6 {  
    static void areaOfRectangle(double length, double width) {  
        double area = length * width;  
        System.out.println("Area = " + area);  
    }  
    public static void main(String[] args) {  
        areaOfRectangle(5.5, 3.2);  
    }  
}
```

4. Methods That Return a Value

```
public class Example7 {  
    static int multiply(int a, int b) {  
        return a * b;  
    }  
    public static void main(String[] args) {  
        int result = multiply(4, 6);  
        System.out.println("Product = " + result);  
    }  
}
```

5. Types of Functions in Java

5.1 Based on Parameters and Return Type

Type	Description	Example
No parameters & no return value	Doesn't take input, doesn't return output.	void sayHello()
Parameters but no return value	Takes input but doesn't return output.	void add(int a, int b)
No parameters but returns value	Doesn't take input but returns a value.	int getNumber()
Parameters and returns value	Takes input and returns a result.	int multiply(int a, int b)

Example — All Four Types Together

```
public class FunctionTypes {  
    // 1. No parameters, no return value  
    static void greet() {  
        System.out.println("Hello!");  
    }  
  
    // 2. Parameters, no return value  
    static void displaySum(int a, int b) {  
        System.out.println("Sum = " + (a + b));  
    }  
  
    // 3. No parameters, returns value  
    static int getLuckyNumber() {  
        return 7;  
    }  
  
    // 4. Parameters and returns value  
    static int multiply(int x, int y) {  
        return x * y;  
    }  
  
    public static void main(String[] args) {  
        greet();  
        displaySum(5, 10);  
        System.out.println("Lucky number = " + getLuckyNumber());  
        System.out.println("Product = " + multiply(3, 4));  
    }  
}
```

5.2 Based on Where They Are Declared

Type	Description	Example
Instance Method	Belongs to an object of a class (called using object name).	obj.display()
Static Method	Belongs to the class itself (called using class name).	ClassName.methodName()
Abstract Method	Declared without implementation — defined in subclasses.	abstract void show();
Final Method	Cannot be overridden by subclasses.	final void display() {}
Native Method	Written in another language (like C/C++), linked to Java.	native void methodName();

Example — Instance vs Static Method

```
public class MethodTypes {
    static void showStatic() {
        System.out.println("I am a static method.");
    }
    void showInstance() {
        System.out.println("I am an instance method.");
    }
    public static void main(String[] args) {
        showStatic(); // called directly, no object needed

        // calling instance method needs an object
        MethodTypes obj = new MethodTypes();
        obj.showInstance();
    }
}
```

5.3 Based on Functionality / Purpose

Type	Description	Example
User-defined methods	Created by the programmer.	void calculateArea()
Predefined methods	Already available in Java libraries (like Math, String).	Math.sqrt(16), "Hello".length()

```
public class FunctionCategory {
    static void userDefined() {
        System.out.println("This is a user-defined method.");
    }
    public static void main(String[] args) {
        userDefined();
        System.out.println("sqrt of 25 = " + Math.sqrt(25));
        System.out.println("Length of word = " + "Java".length());
    }
}
```

6. The static Keyword in Functions

A static method belongs to the class itself, not to any individual object, so it can be called directly using the class name without creating an object.

```
public class Example {  
    static void sayHello() {  
        System.out.println("Hello! I am a static method.");  
    }  
    public static void main(String[] args) {  
        sayHello(); // call directly  
        Example.sayHello(); // or using class name (recommended)  
    }  
}
```

Why Use static in Functions?

Reason	Explanation
1. Call a method without creating an object	Useful for utility/helper functions like Math.sqrt().
2. Share code across all objects	Since it belongs to the class, not to individual objects.
3. Access directly inside main()	main() itself is static — so it can only directly call other static methods.
4. Save memory	Only one copy of the method/variable exists for the entire class.

Wrong Example — Non-static Used Inside a Static Method

```
public class Demo {  
    int x = 10; // instance variable  
  
    static void show() {  
        System.out.println(x); // ERROR: can't access instance variable from a static context  
    }  
}
```

Correct Example

```
public class Demo {  
    static int x = 10; // static variable  
  
    static void show() {  
        System.out.println(x); // OK: static method can access static variable  
    }  
    public static void main(String[] args) {  
        show();  
    }  
}
```

Static vs Non-static Method — Full Example

```
public class MathUtils {  
    // static method  
    static int add(int a, int b) {  
        return a + b;  
    }  
    // non-static (instance) method  
    int multiply(int a, int b) {  
        return a * b;  
    }  
    public static void main(String[] args) {  
        // calling static method - no object needed  
        System.out.println("Sum = " + MathUtils.add(5, 3));  
  
        // calling non-static method - need an object  
        MathUtils obj = new MathUtils();  
        System.out.println("Product = " + obj.multiply(5, 3));  
    }  
}
```

Sum = 8

Product = 15

7. The return Keyword

The return keyword is used inside a method to (1) exit the method, and (2) send a value back to the caller, if the method has a non-void return type.

- If the method's return type is not void, you must return a value matching the declared return type.
- If the return type is void, you can use return; (optional) just to exit the method early.

Example — Method Returning a Value

```
public class ReturnExample1 {  
    static int addNumbers(int a, int b) {  
        int sum = a + b;  
        return sum;  
    }  
    public static void main(String[] args) {  
        int result = addNumbers(5, 10);  
        System.out.println("Sum = " + result);  
    }  
}
```

Output: Sum = 15

Example — Method with void (No Return Value)

```
public class ReturnExample2 {
    static void displayMessage() {
        System.out.println("Hello! This is a void method");
        return; // optional early exit
    }
    public static void main(String[] args) {
        displayMessage();
    }
}
```

Example — Returning Different Data Types

```
public class ReturnExample3 {
    static String greetUser(String name) {
        return "Hello, " + name + "!";
    }
    public static void main(String[] args) {
        String message = greetUser("Alice");
        System.out.println(message);
    }
}
```

Output: Hello, Alice! (a method can return any data type — int, double, String, an object, etc.)

8. Components of a Class

A class in Java is a blueprint that defines how an object behaves and what data it holds. It contains:

- Variables (Data members) — store data
- Methods (Member functions) — define actions
- Constructors — initialize objects
- Blocks (Static / Instance) — execute specific code
- Nested classes — inner classes (optional)

Real-Time Example: A "Car" class represents the blueprint — brand, model, and speed are data; methods like `accelerate()` and `brake()` define behavior. Each car (BMW, Tesla) is an object of that class.

Example 1 — Simple Class

```
class Car {
    String brand = "Tata";
    void start() {
        System.out.println(brand + " is starting...");
    }
}
public class Example1 {
    public static void main(String[] args) {
```

```
    Car c = new Car();
    c.start();
}
}
```

Example 2 — Class with Multiple Methods

```
class Calculator {
    int add(int a, int b) { return a + b; }
    int sub(int a, int b) { return a - b; }
}
public class Example2 {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println(c.add(5, 3));
        System.out.println(c.sub(5, 2));
    }
}
```

Example 3 — Class with Data & Behavior

```
class Student {
    String name = "Asha";
    int marks = 98;
    void display() {
        System.out.println(name + " scored " + marks);
    }
}
public class Example3 {
    public static void main(String[] args) {
        Student s = new Student();
        s.display();
    }
}
```

Example 4 — Class with Constructor & Method

```
class Book {
    String title;
    Book(String t) { title = t; }
    void show() { System.out.println("Book: " + title); }
}
public class Example4 {
    public static void main(String[] args) {
        Book b = new Book("Java Basics");
        b.show();
    }
}
```

Example 5 — Real-Life Example (Bank Account)

```
class Bank {
    String name;
    double balance;
    void deposit(double amount) {
        balance += amount;
    }
    void show() {
        System.out.println(name + " has Rs." + balance);
    }
}
public class Example5 {
    public static void main(String[] args) {
        Bank b = new Bank();
        b.name = "Ravi";
        b.deposit(5000);
        b.show();
    }
}
```

9. Types of Variables in Java

Java has three types of variables in a class:

Type	Declaration Place	Lifetime	Access
Instance Variable	Inside class, outside method	Per object	With object
Static Variable	Inside class with static keyword	Once per class	With class name
Local Variable	Inside method	Until method ends	Inside method only

Real-Time Example — in a company: instance variable = employee name (different for each employee); static variable = company name (same for all); local variable = temporary data like a bonus calculation.

Example 1 — Instance Variable

```
class Employee {
    String name = "Arun";
    int salary = 40000;
    void show() { System.out.println(name + " earns " + salary); }
}
public class Example1 {
    public static void main(String[] args) {
        Employee e = new Employee();
        e.show();
    }
}
```

Example 2 — Static Variable

```
class Employee {
    static String company = "TCS";
    String name;
}
public class Example2 {
    public static void main(String[] args) {
        Employee e = new Employee();
        e.name = "Priya";
        System.out.println(e.name + " works in " + Employee.company);
    }
}
```

Example 3 — Local Variable

```
class Circle {
    void area() {
        double radius = 5;
        double area = 3.14 * radius * radius;
        System.out.println("Area: " + area);
    }
}
public class Example3 {
    public static void main(String[] args) {
        new Circle().area();
    }
}
```

Example 4 — Static and Instance Together

```
class Student {
    static String school = "ABC School";
    String name;
}
public class Example4 {
    public static void main(String[] args) {
        Student s = new Student();
        s.name = "John";
        System.out.println(s.name + " studies at " + Student.school);
    }
}
```

Example 5 — Demonstrating Variable Scope

```
class Test {
    int x = 10;    // instance
    static int y = 20; // static

    void show() {
```

```

        int z = 30; // local
        System.out.println(x + " " + y + " " + z);
    }
}
public class Example5 {
    public static void main(String[] args) {
        new Test().show();
    }
}

```

10. Constructors

A constructor is a special method used to create (initialize) objects in Java. It looks like a method, but:

- Its name is always the same as the class name.
- It does not have a return type (not even void).
- It is automatically called when you create an object using new.

Why we use constructors: to set initial values for an object's data (variables) when it is created.

Real-Life Example: when you buy a mobile, it comes already initialized with a brand and model — just like a constructor initializing data.

Types of Constructors

1. Default Constructor — no parameters, assigns default/fixed values.
2. Parameterized Constructor — takes parameters to assign custom values.
3. Copy Constructor (optional concept) — copies values from another object.

Example — Default Constructor

```

class Student {
    String name;
    int age;

    Student() {
        name = "Unknown";
        age = 18;
        System.out.println("Student object created!");
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        System.out.println(s1.name + " - " + s1.age);
    }
}

```

Student object created!

Unknown - 18

Real-Time Example — Online Shopping Product

Imagine building an app for an online shopping website. Each Product needs a name and a price — a constructor can set these values automatically when a product is created.

```
class Product {
    String name;
    double price;

    Product(String n, double p) {
        name = n;
        price = p;
    }
}

public class Shop {
    public static void main(String[] args) {
        Product p1 = new Product("Laptop", 55000);
        Product p2 = new Product("Phone", 25000);

        System.out.println(p1.name + " costs " + p1.price);
        System.out.println(p2.name + " costs " + p2.price);
    }
}
```

Laptop costs 55000.0

Phone costs 25000.0

Example — Parameterized Constructor

```
class Student {
    String name;
    int age;

    Student(String n, int a) {
        name = n;
        age = a;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Rahul", 20);
        Student s2 = new Student("Priya", 22);

        System.out.println(s1.name + " - " + s1.age);
        System.out.println(s2.name + " - " + s2.age);
    }
}
```

Rahul - 20

Priya - 22

11. Constructor Overloading

Constructor Overloading means having multiple constructors in the same class, but each with a different number or type of parameters. One class can have many constructors that perform different initializations depending on the given input.

Purpose: constructor overloading allows flexibility — you can create an object in different ways.

Rules for Constructor Overloading

- Constructors must have different parameter lists (number or data type).
- The constructor name must be the same as the class name.
- Overloading happens at compile time (called Compile-Time Polymorphism).
- You can call one constructor from another using the `this()` keyword.

Example — Basic Constructor Overloading

```
class Student {
    String name;
    int age;

    Student() {
        name = "Unknown";
        age = 0;
    }

    Student(String n, int a) {
        name = n;
        age = a;
    }

    void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = new Student("Rahul", 21);

        s1.display();
        s2.display();
    }
}
```

Name: Unknown, Age: 0

Name: Rahul, Age: 21

Example — Constructor Overloading (Rectangle)

```
class Rectangle {  
    Rectangle() { System.out.println("Default Rectangle"); }  
    Rectangle(int l, int b) {  
        System.out.println("Area: " + (l * b));  
    }  
}  
public class Example3 {  
    public static void main(String[] args) {  
        new Rectangle();  
        new Rectangle(5, 4);  
    }  
}
```

Example — Overloading with Different Data Types (Box)

```
class Box {  
    double width, height, depth;  
  
    // 1. No-argument constructor  
    Box() {  
        width = height = depth = 0;  
    }  
  
    // 2. One parameter  
    Box(double side) {  
        width = height = depth = side;  
    }  
  
    // 3. Three parameters  
    Box(double w, double h, double d) {  
        width = w;  
        height = h;  
        depth = d;  
    }  
  
    void volume() {  
        System.out.println("Volume = " + (width * height * depth));  
    }  
}  
public class Main {  
    public static void main(String[] args) {  
        Box b1 = new Box();  
        Box b2 = new Box(5);  
        Box b3 = new Box(3, 4, 5);  
    }  
}
```

```

        b1.volume();
        b2.volume();
        b3.volume();
    }
}

```

Volume = 0.0

Volume = 125.0

Volume = 60.0

Example — Constructor Overloading with this() (Student)

The `this` keyword refers to the current object. `this.age = age`; distinguishes between the parameter (`age`) and the object's own field (`this.age`) when they share the same name.

```

class Student {
    int age;
    String name;

    // 1. Default constructor
    Student() {
        age = 18;
        name = "Unknown";
    }

    // 2. Constructor with one parameter
    Student(int age) {
        this.age = age;
        name = "Unknown";
    }

    // 3. Constructor with two parameters
    Student(int age, String name) {
        this.age = age;
        this.name = name;
    }

    void display() {
        System.out.println("Age: " + age + ", Name: " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();           // default
        Student s2 = new Student(20);         // one parameter
        Student s3 = new Student(22, "Arun"); // two parameters
    }
}

```

```
s1.display();
s2.display();
s3.display();
}
}
```

Age: 18, Name: Unknown

Age: 20, Name: Unknown

Age: 22, Name: Arun

Example — Constructor Overloading with this() (Employee, 3 constructors)

```
class Employee {
    int id;
    String name;
    double salary;

    // 1. Default constructor
    Employee() {
        id = 0;
        name = "Not Assigned";
        salary = 0.0;
    }

    // 2. Constructor with two parameters
    Employee(int id, String name) {
        this.id = id;
        this.name = name;
        salary = 15000.0;
    }

    // 3. Constructor with three parameters
    Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    void display() {
        System.out.println("ID: " + id + ", Name: " + name + ", Salary: " + salary);
    }
}

public class Main {
    public static void main(String[] args) {
        Employee e1 = new Employee();
        Employee e2 = new Employee(101, "Kumar");
        Employee e3 = new Employee(102, "Anu", 25000.0);
    }
}
```

```

    e1.display();
    e2.display();
    e3.display();
}
}

```

ID: 0, Name: Not Assigned, Salary: 0.0

ID: 101, Name: Kumar, Salary: 15000.0

ID: 102, Name: Anu, Salary: 25000.0

12. Difference Between Constructor Overloading and Method Overloading

Constructor overloading is used to initialize objects in different ways, whereas method overloading is used to perform different operations using the same method name.

Aspect	Constructor Overloading	Method Overloading
Definition	Multiple constructors in the same class with different parameter lists	Multiple methods in the same class with the same name but different parameters
Purpose	Used to initialize objects in different ways	Used to perform operations in different ways
Name	Constructor name must be same as class name	Method name can be any valid identifier
Return Type	No return type (not even void)	Must have a return type
When called	Automatically called when an object is created	Called explicitly using a method call
this() usage	Can use this() to call another constructor	Cannot use this() to call another method (only this.methodName() is allowed)
Object creation	Object cannot be created without calling a constructor	Object can exist without calling methods
Main role	Object setup / initialization	Business logic / functionality

Example — Constructor Overloading

```

class Demo {
    Demo() {
        System.out.println("Default constructor");
    }
    Demo(int a) {
        System.out.println("Parameterized constructor");
    }
    public static void main(String[] args) {
        Demo d1 = new Demo();
        Demo d2 = new Demo(10);
    }
}

```

Example — Method Overloading

```
class Demo {  
    void show() {  
        System.out.println("No parameters");  
    }  
    void show(int a) {  
        System.out.println("One parameter: " + a);  
    }  
    public static void main(String[] args) {  
        Demo d = new Demo();  
        d.show();  
        d.show(5);  
    }  
}
```

13. Copy Constructor in Java

A Copy Constructor is a special constructor used to create a new object as a copy of an existing object. It copies the values of one object into another object of the same class.

Java does not provide a built-in copy constructor (like C++ does), but you can create one manually.

```
ClassName(ClassName objectName) {  
    // copy each field from objectName into this object  
}
```

Example — Copy Constructor with Modification

```
class Employee {  
    String name;  
    double salary;  
  
    Employee(String n, double s) {  
        name = n;  
        salary = s;  
    }  
  
    // Copy constructor  
    Employee(Employee e) {  
        name = e.name;  
        salary = e.salary;  
    }  
  
    void display() {  
        System.out.println(name + " earns " + salary);  
    }  
}
```

```

public class CopyExample {
    public static void main(String[] args) {
        Employee e1 = new Employee("Asha", 45000);
        Employee e2 = new Employee(e1); // copy constructor used
        e2.salary = 50000; // modify the copy only

        e1.display();
        e2.display(); // proves it's a separate copy
    }
}

```

Asha earns 45000.0

Asha earns 50000.0

Key takeaway: a copy constructor creates a separate, independent copy — changing e2's salary does not affect e1 at all.

Example — Copy Constructor for Complex Data (Book)

```

class Book {
    String title;
    double price;

    Book(String t, double p) {
        title = t;
        price = p;
    }

    Book(Book b) {
        this.title = b.title;
        this.price = b.price;
    }

    void show() {
        System.out.println("Book: " + title + ", Price: " + price);
    }
}

public class CopyConstructorExample {
    public static void main(String[] args) {
        Book b1 = new Book("Java Basics", 499.99);
        Book b2 = new Book(b1);

        b1.show();
        b2.show();
    }
}

```

Example — Copy Constructor (Person)

```
class Person {
    String name;
    Person(String n) { name = n; }
    Person(Person p) { name = p.name; }
}
public class Example4 {
    public static void main(String[] args) {
        Person p1 = new Person("Meena");
        Person p2 = new Person(p1);
        System.out.println(p2.name);
    }
}
```

Example — Constructor with Calculation (Circle)

```
class Circle {
    double area;
    Circle(double r) { area = 3.14 * r * r; }
}
public class Example5 {
    public static void main(String[] args) {
        Circle c = new Circle(5);
        System.out.println("Area: " + c.area);
    }
}
```

14. Ways to Initialize an Object

Objects in Java can be initialized in multiple ways:

Way	Example
1. Using a reference variable	obj.x = 10;
2. Using a constructor	new Student("Asha");
3. Using a method	obj.setDetails(...);
4. Using user input	via Scanner
5. Using another object	obj2 = obj1;

Example 1 — Reference Variable

```
class Student {
    String name;
    int age;
}
public class Example1 {
    public static void main(String[] args) {
        Student s = new Student();
        s.name = "Ravi";
    }
}
```

```
s.age = 20;
System.out.println(s.name + " " + s.age);
}
}
```

Example 2 — Constructor

```
class Student {
    String name;
    int age;
    Student(String n, int a) { name = n; age = a; }
}
public class Example2 {
    public static void main(String[] args) {
        Student s = new Student("Karan", 19);
        System.out.println(s.name + " " + s.age);
    }
}
```

Example 3 — Method

```
class Student {
    String name;
    int age;
    void setDetails(String n, int a) { name = n; age = a; }
    void display() { System.out.println(name + " " + age); }
}
public class Example3 {
    public static void main(String[] args) {
        Student s = new Student();
        s.setDetails("Meena", 18);
        s.display();
    }
}
```

Example 4 — Scanner (User Input)

```
import java.util.Scanner;
class Student {
    String name;
    int age;
    void input() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter name: ");
        name = sc.nextLine();
        System.out.print("Enter age: ");
        age = sc.nextInt();
    }
}
```

```
public class Example4 {
    public static void main(String[] args) {
        Student s = new Student();
        s.input();
        System.out.println(s.name + " " + s.age);
    }
}
```

Example 5 — Object to Object

```
class Student {
    String name;
}
public class Example5 {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.name = "Priya";
        Student s2 = s1; // s2 now refers to the SAME object as s1
        System.out.println(s2.name);
    }
}
```

Important distinction: `Student s2 = s1;` does not create a copy — both `s1` and `s2` point to the exact same object in memory. This is different from a copy constructor, which creates a genuinely separate, independent object.

15. The `this` Keyword in Java

`this` refers to the current object of the class. It is used when local variables (including constructor/method parameters) and instance variables have the same name. `this` can be used in both constructors AND normal methods — it is not only for constructors.

Real-Life Analogy: imagine a classroom where every student has a name tag. When a student says "This is my book," the word "this" refers to that particular student. Similarly in Java, `this` refers to the object that is currently being used.

Why Do We Use the `this` Keyword?

- A method needs access to the entire object, not just one variable.
- We want to send the current object to another class or method.
- We want object-to-object interaction.

Without `this` vs With `this`

Without `this` — if a parameter and an instance variable share the same name, writing `name = name;` inside the constructor just means "local variable = local variable". The instance variable never actually gets a value, because Java resolves the plain name to the closest one in scope, which is the parameter.

```
class Student {
    String name;
    Student(String name) {
```

```
    name = name; // WRONG: instance variable never gets set
  }
}
```

With this — `this.name` refers to the object's own field, while the plain `name` refers to the constructor's parameter, so Java can clearly tell them apart.

```
class Student {
    String name;
    Student(String name) {
        this.name = name; // correct: this.name = object's field, name = parameter
    }
}
```

- Different names -> `this` not required.
- Same names -> `this` required.

Then Why Use the Same Name at All?

Most programmers deliberately use the same names because it makes the code easier to read — it means the parameter name and the class variable name are kept identical, and this is what tells Java which one you mean.

```
class Student {
    String name;
    Student(String studentName) {
        name = studentName; // No need for 'this' - names are different
    }
}
```

This works perfectly because `name` is the instance variable and `studentName` is a clearly different parameter name, so Java can distinguish them without needing `this`.

Using `this` to Refer to Instance Variables

```
class Student {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Ravi", 20);
        s1.display();
    }
}
```

```
}
```

Output: Name: Ravi, Age: 20. Without this, Java would confuse the local (parameter) and instance variables that share the same name.

Using this to Call Current Class Methods

this can also be used to call another method of the current class. It's optional but can be useful for clarity, or when methods are overloaded.

```
class Test {  
    void show() {  
        System.out.println("Hello from show()");  
    }  
    void display() {  
        this.show(); // ensures the current object's show() runs  
        System.out.println("Hello from display()");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Test obj = new Test();  
        obj.display();  
    }  
}
```

Hello from show()

Hello from display()

Using this() to Call Another Constructor

this() is used to call one constructor from another constructor within the same class. It must be the first statement inside the constructor.

```
class Car {  
    String brand;  
    int price;  
  
    Car() {  
        this("Unknown", 0); // calls the parameterized constructor  
        System.out.println("Default constructor called.");  
    }  
  
    Car(String brand, int price) {  
        this.brand = brand;  
        this.price = price;  
    }  
  
    void display() {  
        System.out.println("Brand: " + brand + ", Price: " + price);  
    }  
}
```

```

}
public class Main {
    public static void main(String[] args) {
        Car c1 = new Car(); // calls default constructor
        c1.display();
    }
}

```

*Default constructor called.
Brand: Unknown, Price: 0*

this() helps reuse constructor code — no repetition needed across multiple constructors.

16. Difference Between this in a Method vs this in a Constructor

Aspect	this in Method	this in Constructor
Where it is used	Inside a normal method	Inside a constructor
Meaning of this	Refers to the current object that called the method	Refers to the current object being created
Purpose	Used to access instance variables, pass the current object, or call another method	Used to initialize instance variables or call another constructor
Can pass as argument	Yes - method(this)	Yes - method(this) or new Class(this)
Constructor chaining	Not possible	Possible using this()
When it executes	When the method is explicitly called	Automatically during object creation
Common usage	Business logic, object communication	Object initialization

Example — this in a Method

```

class Student {
    int age = 20;
    void print(Student s) {
        System.out.println("Age: " + s.age);
    }
    void call() {
        print(this); // this refers to the object calling the method
    }
    public static void main(String[] args) {
        Student s = new Student();
        s.call();
    }
}

```

Example — this in a Constructor

```

class Student {
    int age;

```

```

Student(int age) {
    this.age = age; // this refers to the object being created
}
public static void main(String[] args) {
    Student s = new Student(20);
}
}

```

17. Using this to Pass the Current Object as an Argument

this can be passed as a parameter to another method or constructor — this is especially useful in event handling and callbacks, where one object needs to hand a reference to itself to another object.

```

class A {
    void display(B obj) {
        System.out.println("Method called using this keyword.");
    }
}
class B {
    void callMethod() {
        A a = new A();
        a.display(this); // passing the current (B) object
    }
}
public class Main {
    public static void main(String[] args) {
        B obj = new B();
        obj.callMethod();
    }
}

```

Output: Method called using this keyword. — this passes the reference of the current object to another class or method, letting class A interact with the specific B object that called it.

18. Quick Practice Questions — this Keyword

- Write a Java class Employee that uses this keyword to initialize employee name and salary.
- Create a class Book where one constructor calls another using this().
- Write a program where this is used to call current class methods.
- Demonstrate how this can be used to pass an object to another method.
- Create a Laptop class using method chaining (return this).

19. Tips for Understanding Functions, Constructors & this

1. Remember the one hard rule for constructors: same name as the class, no return type at all (not even void). If you see a return type next to a method with the class's name, it's just a regular method, not a constructor.

2. A constructor's job is initialization, not general logic. If you find yourself writing loops or complex calculations inside a constructor, that logic probably belongs in a separate method instead.
3. Static vs instance is really about 'belongs to the class' vs 'belongs to each object'. A static method/variable exists once, shared by everyone. An instance method/variable exists separately for every object you create — this is why static methods can't directly touch instance variables.
4. When overloading (methods or constructors), Java tells calls apart purely by the parameter list (count, order, or types) — the return type or method name alone is never enough to distinguish two overloaded versions.
5. Use `this.fieldName = fieldName;` whenever a constructor or method's parameter has the exact same name as the class's own field. Without this, both sides refer to the parameter, and the actual object field never gets set — this is the single most common beginner mistake with this.
6. Use `this()` (with parentheses) to call one constructor from another in the same class — it must be the very first line in the constructor, and it exists purely to avoid repeating the same initialization code across multiple constructors.
7. Don't confuse a copy constructor with simple assignment. `Employee e2 = new Employee(e1);` creates a brand-new, independent object with copied values. `Employee e2 = e1;` does NOT copy anything — both names just point to the exact same object in memory, so changes through either name affect the same data.
8. `this` can go anywhere an object reference can go — as `this.field` (access a field), `this.method()` (call a method), `this()` (call a constructor), or plain `this` (pass the current object elsewhere, as in `a.display(this)`). It's the same keyword doing four related jobs.
9. A void method can still use `return`; on its own (with no value) — this is a valid way to exit the method early, without producing an output value.
10. Trace constructor calls the same way you'd trace a loop: write down which constructor runs for each new `Object(...)` call in `main()`, and what values get assigned — this is the fastest way to predict a program's output, especially with overloading and `this()` chaining (see the Employee and Car examples above).